GenAI

Interview Question Bank

(Beginner to Production)

80 Questions · 9 Sections

What's Inside?

- Covers fundamentals to advanced & production-level topics
- Real-world scenarios, coding, system design & behavioral Qs
- Essential for GenAI / LLM / RAG / Agent interviews
- Handpicked questions from industry best practices
- Perfect for engineers, builders & aspiring AI professionals

"The best way to predict the future is to build it." — Build. Learn. Ship.

Contents

SECTION 1

Beginner

Q1 – Q10 · Core concepts every GenAI engineer should know

1 What is a Large Language Model and how is it different from traditional ML models?

LLMs are deep learning models (usually transformer-based) trained on vast amounts of text to understand and generate human-like language.

Traditional ML models are usually task-specific, use hand-crafted features, and need more labeled data.

2 What is tokenization and why does it matter?

Tokenization converts text into smaller units (tokens) that models can understand. It affects context length, cost, performance, and how text is represented by the model.

Example — "Hello, world!" → [Hello] [,] [world] [!]

3 What is an embedding and what is it used for?

An embedding is a dense vector representation of text (or other data) in a high-dimensional space. Used for semantic search, clustering, recommendations, similarity, and as input to many NLP systems.

Text → [0.12, -0.45, 0.33, ...] (embedding vector)

4 What is RAG and what problems does it solve?

RAG (Retrieval-Augmented Generation) retrieves relevant documents from a knowledge base and uses them as context for the LLM.

Solves problems like outdated knowledge, hallucinations, and lack of domain-specific information.

Query → Retriever (Vector DB) → Top-k docs → LLM → Answer

5 What is a vector database and how is it different from a traditional database?

A vector DB stores and indexes vector embeddings for similarity search (cosine, dot product). Traditional DBs store structured data (rows/columns) and are optimized for exact matches and transactions.

Vector DB	*Traditional DB*
Finds similar vectors	Finds exact matches

6 What is prompt engineering and why does it matter?

Prompt engineering is the art of crafting inputs (prompts) to get better, more reliable, and more relevant outputs from an LLM.

7 What is the context window and why does it matter?

The context window is the maximum number of tokens the model can "see" in one request (prompt + response). It limits how much information you can provide and affects cost and performance.

System Prompt + Context Window → limited by the model's max token capacity

8 What is hallucination and what causes it?

Hallucination is when the model generates confident but false or unsupported information.

- Missing context
- Ambiguous questions
- Out-of-distribution inputs
- Training data limitations

Example — Model states: "The CEO of Apple in 2021 was ..." (incorrect information)

9 Explain the difference between fine-tuning, RAG, and prompt engineering. When do you use each?

Method	Use When...
Prompt Engineering	No training needed. General-purpose tasks, quick iteration.
RAG	No model training. Need up-to-date or private/domain data.
Fine-tuning	Train the model on your data. Specialized behavior, format control, or when the model must "learn" your domain deeply.

10 What is the difference between encoder-only, decoder-only, and encoder-decoder transformer architectures?

Architecture	How it works	Used for
Encoder-only (e.g., BERT)	Understands input text	Classification, search, embeddings
Decoder-only (e.g., GPT)	Generates text autoregressively (one token at a time)	Generation, chat, code
Encoder-Decoder (e.g., T5)	Encoder understands input, decoder generates output	Translation, summarization

Most LLMs you use daily (ChatGPT, Claude, Llama, Mistral) are decoder-only.

★ *Key takeaway: build systems that are relevant, reliable, safe, and efficient.* ★

SECTION 2

Intermediate

Q11 – Q20 · RAG failure modes, evaluation, agents, and serving

11 Walk me through the failure modes of a naive RAG system and how you fix each one.

Failure Mode	How to Fix
Poor retrieval (irrelevant docs)	Better embeddings, metadata filters, hybrid search
Missing context (relevant docs not retrieved)	Increase recall, query rewrite, hybrid search
Bad chunking (too big/small, splits meaning)	Semantic chunking, overlapping chunks
Hallucinations / unsupported answers	Citations, smaller context window, cite sources
High latency	Caching, smaller model, index optimization
High cost	Compression, response caching, smaller/cheaper models
Data leakage / wrong access control	Proper authZ, metadata filters, tenant isolation

12 What is hybrid search and when is it better than pure semantic search?

Hybrid search combines keyword search (BM25) and semantic (vector) search, then merges the results.

Better when:

- Query has exact terms (codes, names, numbers)
- Spelling variations / shorthand
- Acronyms
- A mix of lexical and conceptual meaning

Query → Keyword Search (BM25) + Semantic Search (Embeddings) → Merge & Rerank → Top Results

13 Explain the difference between LoRA and QLoRA. When would you choose one over the other?

	LoRA	QLoRA
What it is	Low-Rank Adaptation — adds trainable rank-decomposition matrices to attention/FFN layers	Quantized LoRA — base model loaded in 4-bit
Base precision	FP16 / FP32	4-bit, with adapters kept in higher precision (typically 16-bit)
Memory footprint	Higher	Much lower

Choose:

- LoRA: when you have enough GPU memory and need maximum quality.
- QLoRA: when you have limited GPU memory and need cost-efficient training.

14 What are the RAGAS metrics and what does each one tell you about your RAG system?

- Faithfulness: are the answers consistent with the retrieved context?
- Answer Relevance: how relevant is the answer to the question?
- Context Precision: how much of the retrieved information is actually used?
- Context Recall: did the retrieved info cover what was needed?
- Answer Correctness: is the final answer correct compared to ground truth?

15 What is the ReAct agent pattern and how does it work?

ReAct = Reasoning + Acting. The agent alternates between thinking (reasoning) and taking actions (using tools) to answer.

Thought → Action (use tool) → Observation → Thought (reason) → Final Answer

It works as a loop until the agent has gathered enough information to answer.

16 What is LLM-as-judge and what are its known biases?

LLM-as-judge uses an LLM to evaluate model outputs (e.g., relevance, coherence, helpfulness). It's scalable and consistent, but not perfectly accurate.

Known biases:

- Self-enhancement bias (favors outputs similar to its own style)
- Position bias (prefers the first/last option shown)
- Verbosity bias (longer answers preferred)
- Lack of world grounding

17 What is the OWASP LLM Top 10 and which are most relevant to an engineer building a production LLM system?

OWASP LLM Top 10: (1) Prompt Injection, (2) Sensitive Information Disclosure, (3) Supply Chain Vulnerabilities, (4) Data & Model Poisoning, (5) Improper Output Handling, (6) Excessive Agency, (7) System Prompt Leakage, (8) Vector & Embedding Weaknesses, (9) Misinformation, (10) Unbounded Consumption.

Most relevant for RAG engineers: #1, #2, #4, #6, #8 — these have direct impact on data privacy, content safety, and cost control.

18 What is MCP (Model Context Protocol) and what problem does it solve?

MCP Host (LLM App) ⇄ MCP Servers ⇄ Data Sources / Tools / DBs / File Systems

MCP is a standardized protocol for connecting LLMs to external tools, data, and resources.

Problem it solves: previously, every tool needed custom integrations; MCP standardizes how models access resources, making tools easier to build, maintain, and reuse across models.

19 How does vLLM enable high-throughput LLM serving?

- PagedAttention: efficient KV-cache management for long contexts
- Continuous Batching: groups incoming requests dynamically to keep the GPU fully utilized
- Optimized CPU/GPU kernels: faster inference at lower memory cost
- Tensor Parallelism & Multi-GPU: scales across GPUs with minimal overhead
- OpenAI-compatible API: drop-in replacement

Requests → vLLM Engine (Continuous Batching) → Higher Throughput, Lower Latency

20 Explain semantic caching. How does it reduce LLM API costs?

Semantic caching stores responses for semantically similar queries using embeddings, instead of requiring exact string matches.

New Query → Embed → Search cache for similarity (e.g. 0.92 > 0.85 threshold) → Hit → return cached response | Miss → call LLM API & store response

Benefit: fewer repeated/near-duplicate calls to the LLM API → lower cost.

★ *Key takeaway: build systems that are relevant, reliable, safe, and efficient.* ★

SECTION 3

Advanced

Q21 – Q30 · Designing for scale, latency, safety, and failure

21 Your production RAG system is working well on simple queries but failing on multi-hop questions. What's the architecture change?

Use a Multi-Hop / Decomposition + Iterative Retrieval architecture.

> *Query (LLM) → Sub-Query 1 / Sub-Query 2 → Retrieval (x2) → Synthesis → Answer*

Key ideas:

- Break the question into sub-questions
- Retrieve iteratively — each sub-answer feeds the next query
- Maintain a working memory / scratchpad
- Optionally build a knowledge graph or use GraphRAG for entity linking

22 Describe the lost-in-the-middle problem and how you engineer around it.

Lost-in-the-Middle: in long contexts, LLMs tend to pay less attention to information in the middle and focus more on the beginning and end.

> *Risk: important context placed in the middle gets ignored.*

How to engineer around it:

- Put the most important info at the start/end
- Use compression/summarization for large context
- Re-order chunks by relevance (most relevant first/last)

- Use hierarchical retrieval (summary first, details on demand)
- Keep context window utilization moderate (70–80%)

23 How do you design an LLM system for strict latency requirements (sub-800ms response)?

- Use faster models: small/distilled models (e.g., 7B, GPT-3.5-Turbo) or optimized APIs (Groq, together.ai)
- Reduce retrieval: tight top-k (3–5), embedding indexes for fast retrieval, filters to cut search space
- Parallelize: run retrieval and embedding tasks in parallel where possible
- Cache aggressively: warm cache for common queries, semantic cache
- Stream responses: stream tokens as generated for better perceived speed
- Continuously monitor: measure every step and optimize the slowest component

Target budget (sub-800ms): Retrieval < 150ms · Rerank < 100ms · LLM < 400ms · Overhead < 70–150ms

24 How would you build multi-tenancy in a vector DB where different users must only access their own documents?

- Tenant isolation: store tenant_id alongside each embedding
- Filtered retrieval: always filter by tenant_id during retrieval
- Separate indexes / Row-Level Security: enforce RLS at the DB layer (Postgres/pgvector) for docs and data
- Audit & monitoring: log access and use anomaly detection

Golden rule: never rely only on application-level logic — enforce isolation at the data layer.

25 What is the difference between temperature, top-p, and top-k? When would you adjust each?

- Temperature: controls randomness by scaling logits before softmax. Low (0–0.3) → more deterministic/factual. High (0.8–1.2) → more creative/diverse.
- Top-p (nucleus sampling): samples from the smallest set of tokens whose cumulative probability ≥ p. Good balance of creativity and coherence (typical: 0.8–0.95).
- Top-k sampling: samples only from the top-k most likely tokens, limiting the candidate pool (typical: 20–50).

Use low temperature + top-p for factual QA. Use higher temperature or top-k for creative writing or brainstorming.

26 A user's query is routed to your RAG system but the query is ambiguous. How do you handle it?

Detect ambiguity:

- Low-confidence retrieval
- Multiple possible topics
- Vague / under-specified query

Clarify with the user:

- Ask a targeted question or offer options to narrow intent — e.g. "Do you mean …?" or "Which time period are you referring to?"

If clarification fails: fall back to a default (general answer) or route to a human in the loop. Otherwise proceed with the clarified query and continue retrieval and generation.

27 How do you evaluate an agent's performance beyond just "did it complete the task?"

Dimension	What it measures
Task Success	Did it accomplish the goal?
Correctness	Is the final result accurate and grounded?
Efficiency	How many steps, tool calls, cost, and latency?
Robustness	Handles errors, unexpected inputs, retries gracefully?
Safety	No unsafe actions, respects boundaries?

Use a mix of automatic metrics and human evaluation.

28 How do you handle LLM API failures and rate limits in a production application?

- Retry with exponential backoff: handle 429/5xx with jittered backoff
- Circuit breaker: stop calling after repeated failures; fail fast, then recover
- Fallback models: use a smaller/cheaper model as a fallback
- Queue & throttle: rate-limit requests, queue when needed
- Graceful degradation: return cached results, partial answers, or helpful error messages
- Monitoring & alerts: track errors, latency, and rate-limit hits

Best practices: idempotent requests · timeouts · request collapsing · caching · user-friendly errors

29 What is context engineering and how is it different from prompt engineering?

Prompt Engineering	Context Engineering
Focus on how you instruct the model.	Focus on what context you provide.
Crafting prompts, examples, formats, system messages.	Selecting, ordering, compressing, and structuring the right information.
Works on the "instructions" layer.	Works on the "information" layer.

Good context > perfect prompt.

30 How do you handle LLM API failures and rate limits in a production application?

(System design version — infrastructure controls layered on top of the retry logic above.)

Control	Behavior
Retry with Exponential Backoff	Handle 429/5xx with jittered backoff
Circuit Breaker	Stop calling after repeated failures. Fail fast, then recover
Fallback Models	Use a smaller/cheaper model as fallback
Queue & Throttle	Rate-limit requests, queue when needed
Graceful Degradation	Return cached results, partial answers, or helpful error messages
Monitoring & Alerts	Track errors, latency, rate-limit hits

Best practices: idempotent requests · timeouts · request collapsing · caching · user-friendly errors

★ *Advanced systems think ahead, design for failure, and optimize for users.* ★

SECTION 4

Scenario-Based

Q31 – Q35 · Real production situations and how to work through them

31 You shipped a RAG feature last week. This week, user satisfaction dropped 20%. The model and index have not changed. What do you investigate?

Investigate:

- Query distribution shift
- Upstream data changes (sources/APIs)
- Chunking / preprocessing changes
- Retrieval quality (top-k relevance)
- Reranker / prompt / formatting changes
- Latency / timeouts / errors
- UI / UX or answer presentation issues
- Hallucinations / safety incidents
- User segment impacted (new users/orgs?)

How:

- Compare analytics: before vs. after
- Check logs, traces, evaluations
- Sample bad queries and inspect retrieved context + answers
- Check user feedback / tickets
- Run evals on a holdout set

Goal: find what changed in reality, not in code.

32 An enterprise client wants to deploy your LLM application on-premises because of data privacy requirements. What changes?

Key changes:

- Local inference: use on-prem LLMs (e.g., Llama 3, Mistral) via vLLM / TGI / Ollama
- Data stays internal: all data, embeddings, and logs remain on-prem
- Self-hosted components: Vector DB (Qdrant/Weaviate), reranker, monitoring, observability
- Networking: Private VPC/VLAN, firewalls, no outbound internet (or allowlist only)
- Security & compliance: SSO/AD integration, encryption at rest/in transit, audit logs
- DevOps: on-prem CI/CD, model updates, backups, disaster recovery

Trade-off: higher infra cost, more ops responsibility, potentially smaller models.

33 You are given a 500-page legal document and asked to build a Q&A system over it that lawyers can use. What do you build?

Stage	What happens
1. Ingestion	Convert PDF → text; OCR if scanned; extract metadata (sections, headings, page numbers)
2. Chunking	Semantic chunking, ~300-800 tokens with overlap; preserve structure (sections, clauses)
3. Indexing	Create embeddings; store in vector DB; store metadata (section, page, doc id)
4. Retrieval	Top-k semantic search; metadata filters (section, date, etc.); rerank results
5. Generation	Grounded answer with citations; include page/section references; say "I don't know" if context is insufficient
6. UX for Lawyers	Clean Q&A interface; citations with links to sections/pages; export/share answers

Nice-to-have: doc outline sidebar · search within a section · highlighted citations · follow-up questions · versioning & updates

34 Your agent is sending duplicate emails to customers. How do you diagnose and fix it?

Diagnose:

- Check logs: tool calls, inputs, outputs, timestamps
- Look for retries / timeouts causing re-execution
- Verify idempotency: does the email tool prevent duplicates?
- Check state management: is the agent remembering it already sent the email?
- Review prompts & instructions for unclear constraints
- Check external systems (webhook retries, queue duplicates)

Fix:

- Make the email tool idempotent: require a unique message_id (or check recipient + subject + template hash)
- Write confirmations to durable storage (DB) and check before sending
- Add a guardrail in the prompt: "Do not send the same email twice."
- Use a human-in-the-loop for critical sends
- Add a dedup window (e.g., don't resend within N hours)
- Add monitoring & alerts for duplicate actions

35 How would you reduce the cost of a RAG system that currently spends $8,000/month on LLM API calls?

Lever	Tactics
1. Reduce Calls	Add semantic cache for similar queries; route simple queries to a cheaper model/ SLM; use query classification to avoid the LLM when possible
2. Improve Retrieval	Better chunking → fewer irrelevant tokens; metadata filters to shrink candidate set; rerank smaller top-k (e.g., 10 → 5)
3. Optimize Prompts	Shorter system prompts; compress context (summarize retrieved docs); remove verbose examples / text
4. Model Strategy	Use smaller/cheaper models for most tasks; distill/fine-tune a smaller model for your use case; batch requests when real-time isn't required
5. Operational	Monitor token usage; set max tokens & stop early; use streaming to cut failed long responses

Track $/query continuously and optimize the most expensive components first.

★ *Great systems are built by diagnosing the right problem and optimizing continuously.* ★

SECTION 5

Coding

Q36 – Q38 · Practical implementation questions

36 Write a function that calls the OpenAI API with exponential backoff on rate limit errors.

```
import openai
import time
import random
from openai import OpenAI
from openai.error import RateLimitError, APIError, Timeout

client = OpenAI()

def call_openai_with_backoff(messages, model="gpt-4o-mini",
                             max_retries=5, base_delay=1.0, max_delay=60.0):
    attempt = 0
    while True:
        try:
            response = client.chat.completions.create(
                model=model,
                messages=messages,
                timeout=30,
            )
            return response
        except (RateLimitError, Timeout) as e:
            attempt += 1
            if attempt > max_retries:
                raise RuntimeError(f"Max retries ({max_retries}) exceeded. Last
error: {e}")
            delay = min(max_delay, base_delay * (2 ** (attempt - 1)))
            jitter = random.uniform(0, delay * 0.1)
            print(f"[!] Rate limit. Retry in {delay:.2f}s (attempt
{attempt})...")
            time.sleep(delay + jitter)
        except APIError as e:
            # Non-rate-limit API errors: don't retry
            raise e
```

How it works:

- Retries on RateLimitError and Timeout

- Exponential backoff: base_delay × 2^(attempt-1)
- Adds jitter to avoid thundering herd
- Stops after max_retries

Usage:

```
messages = [{"role": "user", "content": "Explain RAG."}]
resp = call_openai_with_backoff(messages)
print(resp.choices[0].message.content)
```

Notes — for production: log errors, use a circuit breaker, and return graceful fallbacks.

37 Write a minimal RAG pipeline that takes a list of text documents and answers a query.

```
from typing import List
from openai import OpenAI
import numpy as np

client = OpenAI()
EMBED_MODEL = "text-embedding-3-small"
CHAT_MODEL = "gpt-4o-mini"
TOP_K = 4

# 1) Embed documents
def embed(texts: List[str]) -> np.ndarray:
    res = client.embeddings.create(model=EMBED_MODEL, input=texts)
    return np.array([d.embedding for d in res.data])  # shape: (n, dim)

# 2) Retrieve top-k docs by cosine similarity
def retrieve(query: str, docs: List[str], doc_embeds: np.ndarray, k: int =
TOP_K) -> List[str]:
    q_emb = np.array(client.embeddings.create(model=EMBED_MODEL,
input=[query]).data[0].embedding)
    sims = doc_embeds @ q_emb  # cosine similarity (OpenAI embeddings are
normalized)
    top_idx = np.argsort(-sims)[:k]
    return [docs[i] for i in top_idx]

# 3) Generate answer
def generate(query: str, context_docs: List[str]) -> str:
    context = "\n".join([f"[{i+1}] {d}" for i, d in enumerate(context_docs)])
    system = "You are a helpful assistant. Use the provided context to answer.
If you don't know, say so."
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    resp = client.chat.completions.create(
        model=CHAT_MODEL,
        messages=[{"role": "system", "content": system}, {"role": "user",
"content": prompt}],
        temperature=0.2,
    )
    return resp.choices[0].message.content.strip()

# 4) Orchestrate
def rag_pipeline(docs: List[str], query: str, k: int = TOP_K) -> str:
    doc_embeds = embed(docs)
    top_docs = retrieve(query, docs, doc_embeds, k)
    return generate(query, top_docs)

# Example
# docs = ["doc1 text...", "doc2 text...", ...]
# print(rag_pipeline(docs, "What is RAG?"))
```

Minimal by design — no external DB, in-memory index (numpy), easy to extend with a persistent vector DB (Qdrant, Pinecone, etc.) and reranking.

38 Write a tool definition for an LLM that searches a database and handles argument validation.

```
from typing import Optional, Literal, Dict, Any
from pydantic import BaseModel, Field, validator
import sqlite3

# ---- Argument schema & validation ----
class SearchDBArgs(BaseModel):
    query: str = Field(..., description="Search keywords or natural language
query")
    table: str = Field(..., description="Table to search", regex="^[a-zA-Z_][a-
zA-Z0-9_]*$")
    limit: int = Field(5, ge=1, le=50, description="Max rows to return (1-50)")
    filters: Optional[Dict[str, Any]] = Field(None, description="Equality
filters {col: value}")
    sort_by: Optional[str] = Field(None, description="Column to sort by")
    order: Literal["asc", "desc"] = Field("asc", description="Sort order")

    @validator("query")
    def query_not_empty(cls, v):
        if not v.strip():
            raise ValueError("query cannot be empty")
        return v

# ---- Tool implementation ----
DB_PATH = "app.db"  # contains tables

def search_database(args: SearchDBArgs) -> Dict[str, Any]:
    # Basic safety: prevent SQL injection via parameterized queries
    allowed_tables = {"customers", "orders", "products", "invoices"}
    if args.table not in allowed_tables:
        raise ValueError(f"Table '{args.table}' is not allowed.")
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    # ... build parameterized SQL from filters/sort_by/order/limit and execute
    cur.execute(query, params)
    rows = cur.fetchall()
    conn.close()
    return {"rows": rows}
```

Key points:

- Validates inputs with Pydantic
- Prevents SQL injection by using parameterized queries
- Restricts to an allowlisted set of tables
- Bounds limit (1–50)
- Returns structured JSON the LLM can use
- Easy to adapt to Postgres/MySQL

Example call from LLM → { "name": "search_database", "arguments": { "query": "acme", "table": "customers", "limit": 10, "filters": {"country": "US"}, "sort_by": "created_at", "order": "desc" } } → Tool Result (JSON)

★ *Good code is not just correct — it's safe, reliable, and easy to maintain.* ★

SECTION 6

Behavioral

Q39 – Q42 · Impact, ownership, and continuous growth

39 Tell me about a time you improved the performance of an AI system you owned.

Situation:

Our RAG chatbot had low answer accuracy (~62%) and users often said answers were incomplete.

Action:

- Analyzed failures: low retrieval recall + poor chunking
- Switched from fixed 500-token chunks to hierarchical chunking (sections + paragraphs)
- Added a cross-encoder reranker (bge-reranker-large)
- Improved query rewriting with HyDE + multi-query
- Introduced a citation requirement in the prompt
- Built an eval set and tracked RAGAS metrics

Result:

- Answer correctness (RAGAS): 62% → 86%
- Hallucination rate: 28% → 9%
- User satisfaction (CSAT): 3.2/5 → 4.5/5
- Support tickets related to wrong answers: ↓ 60%

Key takeaway: data + retrieval quality + evaluation = biggest wins.

40 Describe a time you had to make a tradeoff between model quality and cost in production.

Context:

A summarization feature was costing ~$6K/month on GPT-4. Quality was high, but we needed to optimize.

Option A – High Quality (GPT-4)	Option B – Cost Optimized (GPT-3.5-turbo + Rerank)
Best quality · handles edge cases · high cost · slower	~6× cheaper · faster · slightly lower fluency · needs prompt tuning

What I did:

- Used GPT-3.5-turbo with an improved prompt + length control
- Added a low-cost quality filter (LLM-as-judge) to catch bad outputs
- Escalated only complex inputs to GPT-4 (10–15% of traffic)

Outcome:

- Cost reduced by ~72% ($6K → ~$1.7K/month)
- Quality impact minimal (CSAT 4.3 → 4.1/5)
- Latency improved ~35%

Lesson: optimize for the 80% first, and route the rest.

41 Tell me about a production AI incident you managed. What happened and what did you change?

Incident:

Users reported the chatbot giving confident but incorrect answers after a data source update.

Impact:

- ~18% of answers contained outdated information
- Trust and CSAT dropped
- Support volume increased

Root cause:

Our ingestion job partially failed, so the index had a mix of old and new documents. We had no freshness checks.

What I did:

- Rolled back to last known-good index
- Added data freshness validation + alerts
- Implemented source metadata (timestamp, version)
- Added "last updated" in answers + citations
- Built a canary eval set to detect quality drops automatically

Outcome:

- Issue resolved in ~2 hours
- No repeat incidents since implementing safeguards
- Users regained trust; CSAT recovered within a week

What I changed long-term:

Better monitoring, data validation, and evals in CI/CD for our RAG pipeline.

42 How do you stay current in this field?

- Daily learning (30–45 min): read research papers / arXiv summaries; follow key blogs: OpenAI, Anthropic, LangChain, Vercel AI, Hugging Face, Pinecone
- Hands-on practice: build small projects and experiment with new tools/models; reproduce open-source projects and learn by doing
- Community: engage in Discords, Reddit (r/MachineLearning, r/LocalLLaMA); contribute to open source and share learnings
- Newsletters: subscribe to TLDR AI, Import AI, Latent Space, The Batch; skim weekly to catch trends and new releases
- Continuous reflection: keep a learning journal / changelog; run postmortems on wins & failures
- Conferences & talks: watch talks from NeurIPS, ICML, ACL, PyData, etc.; attend virtual meetups and webinars

Mindset: "Stay curious, ship often, and keep learning in public."

★ Behavioral is about impact, ownership, and continuous growth. Show how you think, build, learn, and make things better. ★

SECTION 7

Model Selection & Infrastructure

Q43 – Q58 · Choosing, serving, and operating models in production

43 When would you choose a 13B open-weight model over a frontier API model like GPT-5?

- Data privacy / compliance (no data leaves your infra)
- Cost at scale (predictable, one-time infra cost vs. usage fee)
- Offline / air-gapped environments
- Customizability (fine-tune, modify, add tools/features)
- Lower latency for local use cases
- Good enough quality for your task (domain-specific)
- You have MLOps capability to run and maintain it

44 What is quantization and how does it affect LLM inference?

Quantization reduces the precision of model weights (e.g., FP32 → INT8/INT4) to save memory and speed up inference.

Effects:

- Memory usage (4–8× less)
- Throughput / faster inference
- Cost (smaller GPUs / more tokens per $)
- Slight quality drop (INT4 > INT8 > FP16 in loss)

FP16 (16-bit) → INT4 (4-bit): smaller footprint, faster inference, minor accuracy trade-off.

45 What is the difference between latency and throughput in LLM serving, and how do you optimize each?

Latency	Throughput
Time to first token / response time. Affects single-user experience.	Tokens/sec or requests/sec. Affects total capacity.

Optimize by:

- Latency: faster model, shorter context, quantization, KV-cache, speculative decoding, continuous batching, better infra (GPU/TPU)
- Throughput: batching, larger context parallelism, tensor/pipeline parallelism, KV-cache reuse, vLLM/TGI, autoscaling

46 What is speculative decoding and when should you use it?

Speculative decoding uses a small draft model to generate multiple tokens ahead, which the large model verifies in parallel.

Use when:

- You need lower latency for long outputs
- You have a good small draft model (e.g., 1–7B)
- You can trade a bit more compute for big speedups (1.5–3×)

47 Explain the difference between HNSW and IVF indexing in vector databases. When does each perform better?

HNSW (Hierarchical Navigable Small World)	*IVF (Inverted File Index)*
Graph-based index · excellent recall · low latency for search · high memory usage · good for dynamic datasets. Best for: high recall + low latency requirements (top-k search).	Cluster-based (k-means + inverted list) · lower memory usage · faster build, better for large-scale · recall depends on nprobe. Best for: very large datasets (millions-billions) where memory is limited and slight recall trade-off is acceptable.

Rule of thumb: HNSW for quality & speed, IVF for scale & memory efficiency.

48 What is prompt compression and how does it reduce cost without sacrificing quality?

Reduce prompt tokens while keeping the essential information.

Techniques:

- Summarization of long docs / chat history
- Extractive compression (keep key sentences/facts)
- Remove redundant / low-importance tokens
- Use semantic filtering + structured representations (tables, JSON)

Fewer input tokens → lower cost + faster latency, while preserving the signal the model needs.

49 How do you handle PII (Personally Identifiable Information) in an LLM application?

- Detect: use PII detection (regex + NER models)
- Minimize: don't collect what you don't need
- Mask/Redact: replace PII with tags (e.g., [NAME], [EMAIL])

- Encrypt: at rest and in transit
- Isolate: private VPC / on-prem / dedicated deployment
- Audit: log access, retain policies, user consent
- Delete: data retention policies + right to be forgotten

50 What is DSPy and how does it differ from traditional prompt engineering?

DSPy (Declarative Self-improving Prompting): a framework to program LM behaviors as modules with signatures (inputs/outputs) and automatically optimize prompts using feedback/data.

Traditional Prompt Engineering	*DSPy*
Manual prompts, ad-hoc iteration	Declarative signatures & modules; auto-optimization (teleprompting)

51 What is a knowledge graph and when does GraphRAG outperform standard vector RAG?

Knowledge Graph (KG): entities (nodes) + relationships (edges), with structured, explicit semantics.

GraphRAG outperforms when:

- Queries need multi-hop reasoning across relationships
- Data is highly structured (people, orgs, products, laws, medical)
- You need explanations with provenance via connected facts
- Handling large schemas / evolving relationships

52 What is the difference between zero-shot, one-shot, and few-shot prompting?

- Zero-shot: no examples. Just instruction + input. Generalization relies on model priors.
- One-shot: one example + input. Sets the pattern.
- Few-shot: several examples + input. Better performance on complex tasks

53 How would you implement A/B testing for LLM prompt changes in production?

- Define metric(s): e.g., task success, CSAT, conversion, latency, cost
- Create variants: Prompt A (control) vs. Prompt B (treatment)
- Traffic split: randomly assign % of users/requests
- Log everything: prompts, inputs, outputs, latency, tokens, cost
- Evaluate with stats: significance tests, guardrail metrics
- Rollout gradually: e.g., 10% → 50% → 100% with kill switch
- Monitor: continually watch dashboard + user feedback

54 What is model distillation and when is it used?

Model distillation trains a smaller "student" model to mimic the outputs of a larger "teacher" model.

Use when:

- You need lower latency / cost
- You want to deploy on edge or smaller GPUs
- You want to protect IP (replace a proprietary model with a smaller one)
- You want to maintain quality with good compression

55 Explain the concept of structured outputs and why they matter for production systems.

Structured outputs ensure the model returns data in a predefined format (JSON, schema, function call, XML).

Why it matters:

- Reliable parsing & automation
- Less post-processing & fewer errors
- Enforces contracts for downstream systems
- Improves safety and consistency

```
{ "name": "Alice", "age": 29, "email": "alice@example.com" }
```

56 What is retrieval with metadata filtering and when is it essential?

Use metadata (e.g., user_id, doc_type, date, language) to filter candidates before/during retrieval.

Essential when:

- Multi-tenant isolation (user/org boundaries)
- Legal/compliance constraints (region, sensitivity)
- Time-aware info (latest docs only)
- Domain-specific subsets (product, department)

57 What is the attention sink phenomenon and how does it affect very long context inference?

Attention sink: the model over-attends to early tokens (or special tokens) and under-weights the middle of long contexts.

Impact: important info in the middle gets lost or under-used, resulting in worse answers.

Mitigation:

- Put critical info at the beginning or end
- Use chunking + retrieval
- Use models with better long-context handling
- Use attention sink tokens (e.g., "sink" tokens / repeat summary)

58 How do you version and manage prompts in production?

- Store prompts in a versioned repository (Git or Prompt SW)
- Use semantic versioning (e.g., v1.2.0)
- Track metadata: description, owner, created_at, variables

- Use templates with variables & defaults
- A/B test and promote: dev → staging → prod
- Rollback capability
- Monitor: track performance per prompt version

Pipeline: Dev v1.3.0 → Staging v1.2.1 → Prod v1.2.0

★ *Choose the right model, run it efficiently, and keep it observable in production.* ★

SECTION 8

Additional Intermediate

Q59 – Q74 · More core concepts, patterns, and trade-offs

59 What is the difference between LangChain and LangGraph? When do you use each?

- LangChain: toolkit for building chains, prompts, retrieval, tools. Great for simple to moderately complex flows.
- LangGraph: graph-based orchestration for complex, stateful, cyclical agent workflows.

Use LangChain for rapid prototyping. Use LangGraph for complex agents with loops, branching, and persistence.

60 What is the "needle in a haystack" test and what does it evaluate?

Evaluates long-context retrieval of a specific fact (the "needle") placed in a large, irrelevant context (the "haystack"). Measures recall across increasing context lengths.

61 What is RLHF and why did DPO largely replace PPO-based RLHF for most post-training?

- RLHF: trains the model using human feedback (rankings) to align behavior.
- DPO (Direct Preference Optimization): simpler, more stable, avoids the reward model + PPO complexity, lower compute, fewer hyperparameters.

62 What is chain-of-thought prompting and when does it help?

Encourages the model to reason step-by-step before answering. Helps on complex reasoning, math, logic, and multi-hop problems.

63 How does an embedding model differ from a generation model architecturally?

- Embedding model: encoder-only (e.g., BERT-style). Outputs dense vectors.
- Generation model: decoder-only or encoder-decoder. Autoregressive token generation.

64 What is the difference between streaming and batch inference for LLMs?

- Streaming: returns tokens incrementally; lower perceived latency; holds the connection open.
- Batch: processes many requests together; higher throughput, better GPU utilization; higher first-token latency.

65 What is a system prompt and what are the security implications of putting sensitive logic in it?

A system prompt sets the model's behavior and rules.

If leaked via jailbreak/prompt injection, attackers can bypass guardrails. Don't put secrets, keys, or critical logic there.

66 What is a reranker and how does it differ from a retriever in a RAG pipeline?

- Retriever: finds relevant documents (recall).
- Reranker: reorders candidates by relevance using a cross-encoder or LLM. Improves precision.

67 What is few-shot prompting via retrieval (also called RAG-based few-shot)?

Retrieve example Q&A pairs similar to the user query and include them in the prompt as in-context examples.

68 What is constitutional AI and how does Anthropic use it to align Claude?

A framework with principles and critiques where the model critiques and revises its own responses. Anthropic trains Claude using this approach alongside other alignment methods.

69 How do you debug an agent that is producing incorrect final answers despite having the correct information in its tool outputs?

- Inspect intermediate steps and tool I/O
- Verify prompt instructions and output format
- Check the reasoning/aggregation step
- Add unit tests, traces, and step-level evals

70 What is the difference between document Q&A and conversational RAG, and how do they require different architectures?

- Document Q&A: single-turn over a static corpus.
- Conversational RAG: multi-turn with chat history, follow-ups, coreference.

Conversational RAG needs memory, history-aware retrieval, and query rewriting.

71 What are the key differences between OpenAI Agents SDK, LangGraph, and CrewAI for production agent development?

- OpenAI Agents SDK: opinionated, tightly integrated with OpenAI models/tools; fastest path to production.
- LangGraph: flexible, low-level control; best for custom, complex workflows.
- CrewAI: role-based multi-agent teams; good for collaborative task execution.

72 What is the chain-of-verification (CoVe) technique and when is it valuable?

The model generates multiple reasoning paths or answers, then verifies/cross-checks them to arrive at the most consistent answer.

Valuable for high-stakes or error-prone tasks.

73 How do you handle a situation where a user's query spans multiple documents that contradict each other?

- Retrieve widely, then rerank

- Surface both sides with sources
- Ask a clarifying question if needed
- Use citation + confidence, let the user decide

74 What is agent memory staleness and how do you manage it?

Staleness = memory contains outdated facts or state.

Manage by: TTLs, refresh policies, source re-checks, versioned memory, summarization with timestamps.

★ *Master the fundamentals. Apply them in context. Ship reliable AI systems.* ★

SECTION 9

System Design Short-Answer

Q75 – Q80 · Whiteboard-style production design questions

75 How would you design a GenAI system to handle traffic spikes without overwhelming the model provider?

- Queue & rate limit: use a request queue (e.g., Redis/SQS) and enforce concurrency & rate limits
- Backpressure: return "queued" status / ETA for non-urgent requests
- Load shedding: drop or defer low-priority traffic during spikes
- Caching: semantic cache for answers; prompt + response cache where possible
- Model tiering: route to smaller/cheaper/faster models when under pressure
- Autoscale workers: scale out stateless workers that pull from the queue
- Circuit breaker: open when error rate/latency crosses thresholds; fail fast or downgrade
- Pre-compute & warm: pre-embed, pre-rank, and keep hot caches warm
- Multi-provider fallback: failover to a secondary provider if primary is throttling/down

76 What metrics would you put on a GenAI product dashboard for a non-technical stakeholder?

- User Impact: active users, queries/conversations, tasks completed
- Success Rate: % successful answers, task completion rate
- User Satisfaction: thumbs up/down %, CSAT
- Time to Value: avg. time to first good answer
- Cost: total cost, cost per 1K queries
- Quality (high level): accuracy/helpfulness trend (using evals or feedback)
- Reliability: uptime, error rate

- Adoption: feature usage, repeat users
- Top Queries/Topics: what users ask most

77 How do you prevent your RAG system from returning stale information?

- Fresh data pipeline: ingest on change (CDC, webhooks) or frequent scheduled syncs
- Document versioning: store versions with timestamps and source URLs
- Recency-aware retrieval: boost recent docs; filter by date when relevant
- TTLs: set expiration for cached results and embeddings if content is time-sensitive
- Source citation: always show source + last-updated date
- Validation step: for critical facts, verify against authoritative APIs/databases
- Monitoring: track a data-freshness lag metric (source update → index update)

78 When would you use pgvector instead of a dedicated vector database like Qdrant?

Use pgvector when:

- Your data already lives in Postgres
- You need strong consistency + ACID transactions with vectors
- Dataset is small-to-medium (up to ~10M vectors depending on hardware)
- You need simple ops (one database to manage)
- You require SQL joins + filters with vector search

Use a dedicated DB (e.g., Qdrant) when:

- You have very large scale (10M+ to billions of vectors)
- You need advanced ANN performance, sharding, replication
- You need hybrid dense/sparse, payload filtering at scale, or multi-tenancy features

79 What is the difference between an LLM gateway and an orchestration framework?

	LLM Gateway	Orchestration Framework
Purpose	Manage access to LLMs	Build and manage complex LLM workflows/ agents
What it does	Routing, load balancing, rate limiting, caching, logging, cost control, model fallbacks	Chains, agents, tools, memory, state, retries, conditional logic, human-in-the-loop
Scope	Closer to infrastructure / platform	Application-level logic and flow
Examples	LiteLLM, Portkey, AI Gateway, Kong AI Gateway	LangGraph, LlamaIndex, CrewAI, Microsoft Autogen

In practice: use a gateway for reliability/cost/control, and an orchestration framework to build the app.

80 How do you build a GenAI feature that needs to degrade gracefully when the LLM API is unavailable?

- Detect early: health checks and circuit breaker

Fallback hierarchy:

- Use a cached answer (semantic/response cache)
- Use a smaller/cheaper local or alternate model
- Use rules/templates/FAQ for simple queries
- Show a helpful message with the option to retry

- Partial functionality: keep non-LLM features working (search, browse, uploads)
- Queue for later: accept the request and process when the service is back
- Communicate clearly: inform users about degraded mode and expected behavior
- Log & alert: track failures and user impact

★ Design for scale. Measure what matters. Degrade gracefully. ★